Problem Set 1 (Challenging)

- **LLM-assisted SAT equivalence checking for structurally divergent circuits**
- **Learning and exploiting invariants for scalable SAT based verification**

Circuit Equivalence Checking using SAT

Problem Statement:

How can **circuit equivalence checking** be performed efficiently using **SAT-based techniques**, especially when circuits undergo transformations or optimizations or between different implementation of same Adder/Multiplier?

- **Use of LLMs to Identify Introduced Comparison Points:** Can **Large Language Models (LLMs)** assist in **identifying appropriate intermediate comparison points** introduced during circuit transformations to improve equivalence checking?
- **Equivalence Between Different Versions of a Design:** How can equivalence be established **between different revisions/versions of the same circuit design**, particularly when structural changes exist?
- **Equivalence Between Multiple Versions of a Circuit:** How can equivalence be verified **across multiple circuit versions of the same module like adder.**
- Can **LLMs be used to learn invariance properties** of circuits that remain unchanged across transformations, and can these invariants aid verification? How can **design-specific (context-aware) invariants** be automatically learned or inferred, rather than relying on generic invariants?

Problem Set 2: (Challenging)

In Context of LLM based verification assertion generation, we have various problem.

- **(i)** **long-context limits**,
- **(ii)** **agent memory**,
- **(iii)** **cloud communication efficiency**,
- **(iv)** **redundancy in multi-agent systems**, and
- **(v)** **feedback-driven refinement of LLM agents**.

This is the detailed that I got from Copilot for these problems.

P1. Long-Context Problem (Partial context, missing relevant info)

LLMs often require more context than can be provided in a single prompt due to token limits, cost, or user constraints. When only partial context is available, LLM outputs can be incomplete, inconsistent, or wrong.

Research Questions

1. **RQ1.1:** Given a task TTT and a large context CCC, how do we select a minimal subset C′⊂CC'\subset CC′⊂C that preserves correctness for TTT?
2. **RQ1.2:** How can the system detect that the provided context is insufficient and request *targeted* missing information (rather than generic "please provide more context")?
3. **RQ1.3:** What representations (summary, structured facts, embeddings, program traces, etc.) best preserve task-critical information under extreme context constraints?

Agentic Memory (Long-term / working memory for agents)

If the user cannot provide all context repeatedly, the system needs **agentic memory**: store → retrieve → update, while avoiding drift and stale knowledge.

Research Questions

1. **RQ1.4:** What memory representation yields best **retrieval fidelity** and **update stability** (text summaries vs structured facts vs embeddings + citations)?
2. **RQ1.5:** How do we prevent **memory poisoning** (incorrect additions), **staleness**, or **contradictions** over time?
3. **RQ1.6:** Can memory be **task-adaptive**, i.e., store different granularity depending on task type (debugging vs writing vs verification)?

Cloud LLM + Efficient Context/Memory Transfer ("Can you send some map?")

When inference happens in the cloud, repeatedly sending long context or full memory is expensive and slow. Need bandwidth-efficient state transfer while preserving correctness.

Research Questions

1. **RQ1.7:** How can we minimize the amount of information sent per request while maintaining answer quality?
2. **RQ1.8:** Which compression schemes are safe for reasoning tasks (lossy summaries vs lossless structured state)?
3. **RQ1.9:** Can we maintain a **server-side session state** securely and still support multi-device or multi-agent access?

Multi-Agent Redundancy

In multi-agent systems, agents often duplicate work, waste tokens/calls, and sometimes produce inconsistent outputs.

Research Questions

1. **RQ2.1:** How do we coordinate agents so they cover tasks **without overlap**, yet preserve diversity when it's useful?
2. **RQ2.2:** How do we detect redundancy early (before cost is incurred) and stop duplicate computations?
3. **RQ2.3:** What task allocation strategy yields best trade-off among quality, cost, and robustness?

Prompt Mapping / Caching to Reduce Network Traffic

Even when agents are coordinated, prompts often share repeated context. If we can reuse or map prompts intelligently, we can reduce network traffic and cost.

Research Questions

1. **RQ2.4:** Can we identify when a new prompt is "close enough" to a previous one such that cached intermediate results can be reused safely?
2. **RQ2.5:** What is the right granularity of caching—prompt-level, chunk-level, tool-call-level, or reasoning-trace-level?
3. **RQ2.6:** How do we validate cached results to prevent error propagation?

Refining the LLM Agent Based on Results

We want the agent to improve over time using outcomes/feedback, but without brittle overfitting or unsafe behavior.

Research Questions

1. **RQ3.1:** What feedback signals are useful and available (user rating, automated tests, downstream success metrics)?
2. **RQ3.2:** How do we turn feedback into improved behavior: prompt optimization, policy updates, tool-use strategy updates, or memory updates?
3. **RQ3.3:** How do we ensure refinement improves performance without degrading generality (catastrophic forgetting) or causing drift?

Problem Set 3:

Background and Motivation

Formal verification relies heavily on **assertions and properties** (e.g., SystemVerilog Assertions – SVA) to specify correctness. Writing high-quality assertions is:

- time-consuming,
- error-prone,
- requires deep design understanding.

Recent advances in **Large Language Models (LLMs)** suggest that they can assist in **automatically generating candidate assertions** from RTL code and/or natural-language specifications. However, LLM-generated assertions may be incomplete, redundant, or even unsound.

The goal of this project is to **systematically study how LLMs can assist assertion generation**, and how **formal validation** can be used to filter, refine, and verify those assertions.

2. Problem Statement

Design and evaluate a system that:

Automatically generates verification assertions using an LLM and validates them using formal verification techniques.

The system must clearly separate:

- **Generation (ML/LLM)** and
- **Validation (formal methods)**

3. Project Objectives

Each project should aim to:

1. **Generate candidate assertions** from RTL designs using an LLM
2. **Classify and formalize assertions** (safety, liveness, invariants)
3. **Validate assertions** using simulation or formal tools
4. **Analyze correctness, usefulness, and limitations** of LLM-generated assertions

4. Input / Output Specification

Inputs

- RTL designs (Verilog/SystemVerilog):
  - FIFO
  - Arbiter
  - FSM controller
  - Simple pipelined datapath
- Optional:
  - Natural-language description of the design

Outputs

- A set of **SystemVerilog Assertions (SVA)**
- Classification of assertions:
  - valid / invalid
  - trivial / useful
  - redundant / unique
- Formal verification results:
  - PASS / FAIL
  - counterexamples (if any)

5. Minimum Required Tasks (Baseline)

Task 1: Assertion Generation using LLM

Students must:

- Use an LLM (ChatGPT / Claude / open-source LLM) to generate assertions
- Prompt examples:
  - "Generate safety assertions for this Verilog module"
  - "What invariants must always hold for this FSM?"

Deliverable:

- Prompts used
- Raw generated assertions

Task 2: Assertion Formalization

Students must:

- Convert LLM output into **syntactically correct SVA**
- Map each assertion to its intent:
  - Safety property
  - Liveness property
  - Functional invariant

Deliverable:

- Annotated list of assertions

Task 3: Formal Validation

Students must:

- Verify the assertions using:
  - Formal tools (SymbiYosys / JasperGold / VC Formal), or
  - Bounded model checking, or
  - Simulation-based checking

Deliverable:

- Verification results
- At least **one counterexample** for a failing assertion

Task 4: Critical Analysis

Students must answer:

- Which assertions are correct but **trivial**?
- Which assertions are **incorrect or over-restrictive**?
- Which assertions reveal **bugs or corner cases**?

5. Research-Oriented Objective

High-performing teams may explore one or more of the following.

A. Assertion Refinement Loop

Implement a loop:

1. Generate assertion
2. Run formal check
3. Feed counterexample back to LLM
4. Refine assertion

Research Question:
Can LLMs improve assertion quality using counterexamples?

B. Invariant Mining + LLM

Combine:

- Simple invariant mining (dynamic simulation traces)
- LLM-generated invariants

Compare: mined vs LLM invariants

C. Redundancy Detection

Identify:

- Equivalence between assertions
- Subsumed or implied properties

Use:

- SAT checks
- Manual reasoning

D. Context Sensitivity

Study impact of:

- RTL-only prompts
- RTL + natural language spec
- RTL + waveforms

Question: Which context yields the best assertions?

9. Constraints and Rules

- LLMs must **not be treated as oracles**
- All generated assertions **must be formally checked**
- Any failing assertion must be analyzed
- Proper citation of LLM usage is mandatory

10. Deliverables

1. **Project report (8–10 pages)**
2. Source code / scripts
3. Prompt logs and LLM outputs
4. Assertion files (SVA)
5. Verification logs

Problem Statement 4

LLM-Guided Decision Heuristics for SAT-Based Automatic Test Pattern Generation

---

1. Background

Automatic Test Pattern Generation (ATPG) is a fundamental problem in digital circuit testing. Modern ATPG techniques increasingly rely on Boolean Satisfiability (SAT) solvers to generate test vectors for structural fault models such as stuck-at and transition faults.

In SAT-based ATPG, the test generation problem is encoded as a CNF formula:

- A fault-free circuit model
- A faulty circuit model
- A miter enforcing output divergence

A SAT solver is then used to determine whether a test vector exists.

While SAT-based ATPG has shown strong performance improvements over classical algorithms (e.g., PODEM, FAN), the efficiency of SAT solving heavily depends on internal heuristics such as:

- Decision variable ordering
- Clause learning
- Branching strategies

These heuristics are purely syntactic and unaware of circuit semantics.

---

2. Problem Gap

Current SAT-based ATPG approaches:

- Treat each fault independently
- Use generic SAT heuristics (e.g., VSIDS)
- Do not leverage structural or functional circuit knowledge beyond CNF encoding
- Do not learn semantic patterns across faults

Large Language Models (LLMs), however, are capable of reasoning over:

- Circuit structure

- Gate-level dependencies
- Sensitization paths
- Functional behavior descriptions

There is currently little to no systematic study on:

Whether LLMs can serve as semantic heuristic oracles to guide SAT-based ATPG search and improve solving efficiency.

---

3. Problem Statement

The objective of this project is to design, implement, and evaluate an LLM-guided SAT-based ATPG framework where the LLM assists the SAT solver by providing structural decision heuristics for fault activation and propagation.

Specifically, the project must address the following research questions:

1. Can an LLM predict useful partial assignments for fault activation and propagation?
2. Can LLM-guided variable selection reduce SAT solver:
    - Decision count?
    - Conflict count?
    - Solving time?
3. Does LLM guidance improve detection of hard-to-detect faults?
4. Can knowledge transfer across previously solved faults improve performance on new faults?

---

4. Proposed Approach

Students must implement the following architecture:

Step 1: SAT-Based ATPG Core

- Convert RTL to gate-level netlist (using Yosys or equivalent)
- Encode stuck-at faults in CNF
- Construct good/faulty circuit miter
- Solve using a SAT solver (e.g., MiniSAT / PySAT / Z3)

Step 2: LLM Guidance Layer

For each fault instance, the LLM may provide one or more of the following:

- Predicted fault activation conditions
- Suggested sensitization paths
- Partial input assignments
- Variable ordering hints

These hints must be translated into:

- Assumption literals
- Branching priority modifications
- CNF constraints (soft guidance)

Step 3: Closed-Loop Evaluation

If SAT fails or conflicts heavily:

- Extract UNSAT core or conflict information
- Provide structured feedback to LLM
- Refine guidance iteratively

---

5. Expected Novelty

The novelty of this project lies in:

1. Introducing semantic reasoning from LLMs into SAT-based ATPG decision processes.
2. Formally evaluating LLM impact on solver complexity metrics.
3. Studying hybrid semantic-structural heuristics in ATPG.
4. Investigating cross-fault knowledge reuse via LLM interaction.

This is distinct from classical SAT-ATPG work, which relies solely on algorithmic heuristics internal to the solver.

---

6. Experimental Evaluation

Students must evaluate their framework on benchmark circuits such as:

- ISCAS85 / ISCAS89
- Arithmetic modules (ALU, multipliers)

Metrics to report:

- Fault coverage
- SAT solving time
- Number of decisions
- Conflict count
- Pattern count
- Comparison with baseline SAT-ATPG

Statistical comparison must be included.

---

7. Deliverables

1. SAT-based ATPG implementation
2. LLM integration module
3. Experimental evaluation report
4. Complexity analysis
5. Discussion on scalability and limitations

Bonus (optional):

- Extension to transition faults
- LLM-guided test point insertion
- Learning across multiple circuits

---

8. Expected Learning Outcomes

Students will gain:

- Practical understanding of SAT-based ATPG
- Experience in CNF encoding of hardware circuits
- Insight into solver heuristics
- Critical evaluation of AI-assisted formal reasoning
- Ability to distinguish hype from measurable algorithmic improvement

---

If you would like, I can also:

- Convert this into a polished PDF-ready project handout
- Add grading rubric (research depth vs implementation vs evaluation)
- Or provide a weaker B.Tech version and a stronger M.Tech/PhD version

This has the potential to be a genuinely strong formal methods course project.